

qwen_saeed_stargazer_eval

June 10, 2026

1 Qwen/Saeed STARGAZER Trajectory-Level Evaluation

This notebook evaluates the completed `qwen_saeed_agent_stargazer.ipynb` run as a trajectory-level scientific-agent trial, aligned with `NLP_Lab___Project_Proposal.pdf`.

The evaluation target is not only the final STARGAZER score. The proposal asks whether trajectory instrumentation can localise, classify, and predict failures that output-only scoring misses. This notebook therefore evaluates:

- **O1 localisation:** event graph, anchor labels, first violated anchor, originating agent/event;
- **O2 failure taxonomy and detectors:** alias convergence, perseveration, format fragility, coordination collapse, cross-agent error propagation, silent overconfidence / critic-masking;
- **O3 early-warning limits:** whether trajectory signals degraded before the final scientific failure was visible;
- **STARGAZER correctness:** final artifact/schema, planet count, period/mass/phase/model-fit anchors.

Implementation note: this notebook intentionally does **not** import from `cambagent_eval`. The benchmark and trajectory-analysis logic below is copied/adapted locally from the project evaluator and rewritten for this single Qwen/Saeed run.

```
[1]: from __future__ import annotations

from collections import Counter, defaultdict
from pathlib import Path
import json
import math
import re
import sys

import pandas as pd
from IPython.display import Markdown, display

ROOT = Path(r"C:\Users\Anwender\Science-Work-Flow-")
RUN_OUT = ROOT / "outputs" / "qwen_saeed_agent_stargazer"
EVAL_OUT = ROOT / "outputs" / "qwen_saeed_stargazer_eval"
EVAL_OUT.mkdir(parents=True, exist_ok=True)

STARGAZER_PACKAGE_ROOT = ROOT / "data" / "stargazer_repo"
```

```

TASK_JSON = STARGAZER_PACKAGE_ROOT / "stargazer" / "Stargazer_real_data_task" / "real_001.json"

FILES = {
    "transition": RUN_OUT / "agent_transition_trace.json",
    "qwen_trace": RUN_OUT / "qwen_trace_full.json",
    "code_reviews": RUN_OUT / "code_review_decisions.json",
    "result_reviews": RUN_OUT / "result_review_decisions.json",
    "executors": RUN_OUT / "executor_records.json",
    "final_verdict": RUN_OUT / "finding_final_verdict.json",
    "benchmark": RUN_OUT / "separate_stargazer_benchmark_judgment.json",
    "planner": RUN_OUT / "planner_summary.txt",
}

def load_json(path: Path, default=None):
    if not path.exists():
        return default
    return json.loads(path.read_text(encoding="utf-8"))

def write_json(path: Path, payload):
    path.write_text(json.dumps(payload, indent=2), encoding="utf-8")

missing = [name for name, path in FILES.items() if not path.exists()]
assert not missing, "Missing run artifacts: " + ", ".join(missing)

transition_records = load_json(FILES["transition"], [])
qwen_trace = load_json(FILES["qwen_trace"], [])
code_reviews = load_json(FILES["code_reviews"], [])
result_reviews = load_json(FILES["result_reviews"], [])
executor_records = load_json(FILES["executors"], [])
final_verdict = load_json(FILES["final_verdict"], {})
cached_benchmark = load_json(FILES["benchmark"], {})
planner_summary = FILES["planner"].read_text(encoding="utf-8")

pd.set_option("display.max_colwidth", 160)
pd.set_option("display.width", 180)

display(Markdown(
    f"Loaded Qwen/Saeed run artifacts from `{RUN_OUT}`. "
    f"Loop stop reason: **{final_verdict.get('loop_stop_reason')}**; "
    f"benchmark ran: **{final_verdict.get('benchmark_ran')}**."
))

```

Loaded Qwen/Saeed run artifacts from C:\Users\Anwender\Science-Work-Flow-\outputs\qwen_saeed_agent_
 Loop stop reason: **APPROVE_RESULT**; benchmark ran: **True**.

1.1 1. Local STARGAZER Benchmark Code

This cell embeds the benchmark/scoring helpers directly in the notebook. It imports only the STARGAZER package for the benchmark itself, not `cambagent_eval`.

```
[2]: REWARD_WEIGHTS = {"likelihood": 1.0, "delta_bic": 0.3, "neg_rms": 0.1, "match": 1.0, "count": 0.2}

def angle_distance(a, b) -> float:
    try:
        raw = abs(float(a) - float(b))
    except Exception:
        return math.inf
    return min(raw % (2.0 * math.pi), (2.0 * math.pi - raw) % (2.0 * math.pi))

def relative_error(observed, expected) -> float:
    try:
        obs = abs(float(observed))
        exp = abs(float(expected))
    except Exception:
        return math.inf
    return abs(obs - exp) / max(exp, 1e-9)

def compact_planet_fields(planet):
    if not planet:
        return {}
    result = {}
    for key in ["P_days", "m_sin_i_mjup", "e", "omega_rad", "l_rad"]:
        value = planet.get(key)
        result[key] = round(float(value), 6) if isinstance(value, (int, float))
    else value
    return result

def prediction_truth_rows(submission, truth_planets, pairs, matching):
    submitted = [planet for planet in submission.get("planets", []) if
    isinstance(planet, dict)]
    rows = []
    seen_truth, seen_guess = set(), set()
    for pair in pairs:
        truth_index, guess_index = int(pair[0]), int(pair[1])
        if truth_index >= len(truth_planets) or guess_index >= len(submitted):
            continue
        truth = truth_planets[truth_index]
        guess = submitted[guess_index]
        seen_truth.add(truth_index)
        seen_guess.add(guess_index)
        rows.append({
            "row_type": "matched",
```

```

        "truth_index": truth_index,
        "guess_index": guess_index,
        "truth": compact_planet_fields(truth),
        "submission": compact_planet_fields(guess),
        "period_rel_error": round(relative_error(guess.get("P_days"), truth.
↪get("P_days")), 6),
        "mass_rel_error": round(relative_error(guess.get("m_sin_i_mjup"),
↪truth.get("m_sin_i_mjup")), 6),
        "eccentricity_abs_error": round(abs(float(guess.get("e", 0.0) or 0.
↪0) - float(truth.get("e", 0.0) or 0.0)), 6),
        "omega_error_rad": round(angle_distance(guess.get("omega_rad", 0.
↪0), truth.get("omega_rad", 0.0)), 6),
        "l_error_rad": round(angle_distance(guess.get("l_rad", 0.0), truth.
↪get("l_rad", 0.0)), 6),
    })
    unmatched_truth = sorted(set(int(i) for i in matching.
↪get("unmatched_truth", [])) | (set(range(len(truth_planets))) - seen_truth))
    unmatched_guess = sorted(set(int(i) for i in matching.
↪get("unmatched_guess", [])) | (set(range(len(submitted))) - seen_guess))
    for truth_index in unmatched_truth:
        if truth_index < len(truth_planets):
            rows.append({"row_type": "unmatched_truth", "truth_index":
↪truth_index, "guess_index": None, "truth":
↪compact_planet_fields(truth_planets[truth_index]), "submission": {}})
        for guess_index in unmatched_guess:
            if guess_index < len(submitted):
                rows.append({"row_type": "unmatched_submission", "truth_index":
↪None, "guess_index": guess_index, "truth": {}, "submission":
↪compact_planet_fields(submitted[guess_index])})
    return rows

def nearest_truth_rows(submission, truth_planets):
    submitted = [planet for planet in submission.get("planets", []) if
↪isinstance(planet, dict)]
    rows = []
    for guess_index, guess in enumerate(submitted):
        if not truth_planets:
            continue
        best_truth_index, best_truth = min(
            enumerate(truth_planets),
            key=lambda item: relative_error(guess.get("P_days"), item[1].
↪get("P_days")),
        )
        rows.append({
            "guess_index": guess_index,
            "nearest_truth_index": best_truth_index,

```

```

        "submission": compact_planet_fields(guess),
        "nearest_truth": compact_planet_fields(best_truth),
        "period_rel_error": round(relative_error(guess.get("P_days"),
↪best_truth.get("P_days")), 6),
        "mass_rel_error": round(relative_error(guess.get("m_sin_i_mjup"),
↪best_truth.get("m_sin_i_mjup")), 6),
        "eccentricity_abs_error": round(abs(float(guess.get("e", 0.0) or 0.
↪0) - float(best_truth.get("e", 0.0) or 0.0)), 6),
        "omega_error_rad": round(angle_distance(guess.get("omega_rad", 0.
↪0), best_truth.get("omega_rad", 0.0)), 6),
        "l_error_rad": round(angle_distance(guess.get("l_rad", 0.0),
↪best_truth.get("l_rad", 0.0)), 6),
    })
    return rows

def stargazer_component_breakdown(submission, truth_planets, pairs, matching,
↪match_score, delta_bic):
    submitted = [planet for planet in submission.get("planets", []) if
↪isinstance(planet, dict)]
    failed = []
    count_ok = len(submitted) == len(truth_planets)
    if not count_ok:
        failed.append("planet_count")
    period_ok_count = mass_ok_count = phase_ok_count = 0
    pair_rows = []
    for pair in pairs:
        truth_index, guess_index = int(pair[0]), int(pair[1])
        if truth_index >= len(truth_planets) or guess_index >= len(submitted):
            continue
        truth, guess = truth_planets[truth_index], submitted[guess_index]
        period_rel_error = relative_error(guess.get("P_days"), truth.
↪get("P_days"))
        mass_rel_error = relative_error(guess.get("m_sin_i_mjup"), truth.
↪get("m_sin_i_mjup"))
        omega_error = angle_distance(guess.get("omega_rad", 0.0), truth.
↪get("omega_rad", 0.0))
        l_error = angle_distance(guess.get("l_rad", 0.0), truth.get("l_rad", 0.
↪0))
        eccentricity_abs_error = abs(float(guess.get("e", 0.0) or 0.0) -
↪float(truth.get("e", 0.0) or 0.0))
        period_ok = period_rel_error <= 0.10
        mass_ok = mass_rel_error <= 0.50
        phase_ok = omega_error <= 1.0 and l_error <= 1.0 and
↪eccentricity_abs_error <= 0.15
        period_ok_count += int(period_ok)
        mass_ok_count += int(mass_ok)

```

```

    phase_ok_count += int(phase_ok)
    pair_rows.append({
        "truth_index": truth_index,
        "guess_index": guess_index,
        "period_rel_error": round(period_rel_error, 6),
        "mass_rel_error": round(mass_rel_error, 6),
        "omega_error_rad": round(omega_error, 6),
        "l_error_rad": round(l_error, 6),
        "eccentricity_abs_error": round(eccentricity_abs_error, 6),
        "period_ok": period_ok,
        "mass_ok": mass_ok,
        "phase_or_eccentricity_ok": phase_ok,
    })
    truth_count = len(truth_planets)
    period_fraction = period_ok_count / truth_count if truth_count else 1.0
    mass_fraction = mass_ok_count / truth_count if truth_count else 1.0
    phase_fraction = phase_ok_count / truth_count if truth_count else 1.0
    if period_fraction < 1.0:
        failed.append("period_recovery")
    if mass_fraction < 1.0:
        failed.append("mass_amplitude")
    if phase_fraction < 1.0:
        failed.append("phase_or_eccentricity")
    if match_score < 0.8 or delta_bic <= 0.0:
        failed.append("model_fit")
    return {
        "failed_components": sorted(set(failed)),
        "planet_count": {"submitted": len(submitted), "truth":
↪ len(truth_planets), "ok": count_ok, "unmatched_guess": matching.
↪ get("unmatched_guess", []), "unmatched_truth": matching.
↪ get("unmatched_truth", [])},
        "period_recovery_fraction": round(period_fraction, 6),
        "mass_recovery_fraction": round(mass_fraction, 6),
        "phase_recovery_fraction": round(phase_fraction, 6),
        "matched_pair_diagnostics": pair_rows,
    }

def task_difficulty_metadata(task):
    times = [float(value) for value in getattr(task.observations, "times_days",
↪ [])]
    sigmas = [float(value) for value in getattr(task.observations, "sigmas_ms",
↪ [])]
    instruments = [str(value) for value in getattr(task.observations,
↪ "instruments", [])]
    truth_count = len(getattr(task.config, "planets", []))
    observation_count = len(times)
    baseline_days = max(times) - min(times) if times else 0.0

```

```

median_sigma = sorted(sigmas)[len(sigmas) // 2] if sigmas else None
stargazer_truth_difficulty = int(getattr(task, "truth_difficulty", 0) or 0)
return {
    "bucket": "easy",
    "factors": {
        "truth_planet_count": truth_count,
        "observation_count": observation_count,
        "baseline_days": round(baseline_days, 6),
        "median_sigma_ms": round(float(median_sigma), 6) if median_sigma is
↪not None else None,
        "instrument_count": len(set(instruments)),
        "stargazer_truth_difficulty": stargazer_truth_difficulty,
    },
}

def evaluate_stargazer_benchmark_local(submission_path: Path, task_json: Path):
    if not submission_path.exists():
        return {"type": "stargazer_benchmark", "path": str(submission_path),
↪"evaluatable": False, "passed": False, "score": 0.0, "criteria":
↪{"file_exists": False}}
    if str(STARGAZER_PACKAGE_ROOT) not in sys.path:
        sys.path.insert(0, str(STARGAZER_PACKAGE_ROOT))
    from stargazer import Task, evaluate_submission

    task = Task.from_json(task_json.read_text(encoding="utf-8"))
    submission = json.loads(submission_path.read_text(encoding="utf-8"))
    truth_planets = [planet.__dict__ for planet in task.config.planets]
    reward, info = evaluate_submission(task.config, task.observations,
↪submission, task.config.planets, reward_weights=REWARD_WEIGHTS,
↪mode="params_and_model")
    matching = info.get("matching", {}).get("assignment", {})
    pairs = matching.get("pairs", [])
    truth_count = len(task.config.planets)
    guess_count = len(submission.get("planets", []))
    matched_truth_fraction = len(pairs) / truth_count if truth_count else 1.0
    match_score = float(info.get("components", {}).get("match", 0.0))
    delta_bic = float(info.get("components", {}).get("delta_bic", 0.0))
    rms = float(info.get("residuals", {}).get("rms", float("nan")))
    criteria = {
        "file_exists": True,
        "planet_count_matches": guess_count == truth_count,
        "matched_truth_fraction_positive": matched_truth_fraction > 0.0,
        "match_score_at_least_0_8": match_score >= 0.8,
        "delta_bic_positive": delta_bic > 0.0,
    }
    return {
        "type": "stargazer_benchmark",

```

```

        "path": str(submission_path),
        "evaluable": True,
        "passed": all(criteria.values()),
        "score": round(sum(1 for ok in criteria.values() if ok) /
        ↪len(criteria), 6),
        "criteria": criteria,
        "reward": round(float(reward), 6),
        "match_score": round(match_score, 6),
        "rms": round(rms, 6) if math.isfinite(rms) else None,
        "delta_bic_per_point": round(delta_bic, 6),
        "submitted_planets": submission.get("planets", []),
        "truth_planets": truth_planets,
        "submitted_planet_count": guess_count,
        "truth_planet_count": truth_count,
        "difficulty_metadata": task_difficulty_metadata(task),
        "matched_truth_fraction": round(matched_truth_fraction, 6),
        "component_breakdown": stargazer_component_breakdown(submission,
        ↪truth_planets, pairs, matching, match_score, delta_bic),
        "prediction_truth_rows": prediction_truth_rows(submission,
        ↪truth_planets, pairs, matching),
        "nearest_truth_rows": nearest_truth_rows(submission, truth_planets),
        "matching_summary": {"matched_pairs": pairs, "unmatched_guess":
        ↪matching.get("unmatched_guess", []), "unmatched_truth": matching.
        ↪get("unmatched_truth", [])},
    }

```

1.2 2. Recompute Output-Level STARGAZER Evidence

The benchmark is intentionally run only in the evaluation notebook, after the agent workflow produced a final executor submission.

```

[3]: final_submission_path = Path(final_verdict.get("final_submission_path", ""))
    submission_artifact_available = final_submission_path.exists()
    submission_path_is_agent_workflow = str(final_submission_path).
    ↪startswith(str(RUN_OUT / "agent_workflow"))

    if submission_artifact_available:
        assert submission_path_is_agent_workflow, "Submission is not from the agent
        ↪workflow directory."
        benchmark = evaluate_stargazer_benchmark_local(final_submission_path,
        ↪TASK_JSON)
        benchmark_source = "recomputed_from_submission"
    else:
        benchmark = dict(cached_benchmark)
        benchmark_source = "saved_benchmark_json_submission_file_missing"
        assert benchmark.get("evaluable") is True, "No final submission file and no
        ↪evaluable saved benchmark record."

```



```

if "nearest_truth_rows" not in benchmark:
    benchmark["nearest_truth_rows"] = nearest_truth_rows(
        {"planets": benchmark.get("submitted_planets", [])},
        benchmark.get("truth_planets", []),
    )
benchmark["benchmark_source"] = benchmark_source
benchmark["submission_artifact_available_at_eval_time"] = ☐
↳ submission_artifact_available
write_json(EVAL_OUT / "local_stargazer_benchmark.json", benchmark)

summary_rows = [
    ["submission_path", str(final_submission_path)],
    ["benchmark_source", benchmark_source],
    ["submission_artifact_available_at_eval_time", ☐
↳ submission_artifact_available],
    ["evaluable", benchmark.get("evaluable")],
    ["passed", benchmark.get("passed")],
    ["score", benchmark.get("score")],
    ["reward", benchmark.get("reward")],
    ["match_score", benchmark.get("match_score")],
    ["matched_truth_fraction", benchmark.get("matched_truth_fraction")],
    ["rms", benchmark.get("rms")],
    ["delta_bic_per_point", benchmark.get("delta_bic_per_point")],
    ["submitted_planet_count", benchmark.get("submitted_planet_count")],
    ["truth_planet_count", benchmark.get("truth_planet_count")],
]
display(pd.DataFrame(summary_rows, columns=["field", "value"]))
display(pd.DataFrame(benchmark.get("prediction_truth_rows", [])))
display(pd.DataFrame(benchmark.get("nearest_truth_rows", [])))

```

	field	
↳		☐
↳	value	☐
0	submission_path C:	
↳	↳\Users\Anwender\Science-Work-Flow-\outputs\qwen_saeed_agent_stargazer\agent_workflow\iterati	
↳	↳json	
1	benchmark_source	☐
↳	↳	☐
↳	↳saved_benchmark_json_submission_file_missing	
2	submission_artifact_available_at_eval_time	☐
↳		☐
↳	False	
3	evaluable	☐
↳		☐
↳	True	

```

4                                     passed                                ❏
↳
↳ False                                                                    ❏
5                                     score                                ❏
↳
↳ 0.4                                                                    ❏
6                                     reward                                ❏
↳
↳ -335387.464489                                                         ❏
7                                     match_score                          ❏
↳
↳ 0.0                                                                    ❏
8                                     matched_truth_fraction               ❏
↳
↳ 0.0                                                                    ❏
9                                     rms                                    ❏
↳
↳ 354.505911                                                             ❏
10                                    delta_bic_per_point                   ❏
↳
↳ -418569.316306                                                         ❏
11                                    submitted_planet_count                ❏
↳
↳ 1                                                                      ❏
12                                    truth_planet_count                   ❏
↳
↳ 1                                                                      ❏

      row_type  truth_index  guess_index                                ❏
↳
      truth \
0  unmatched_truth          0.0          NaN  {'P_days': 4.230785, ❏
↳ 'm_sin_i_mjup': 0.461, 'e': 0.013, 'omega_rad': 1.012291, 'l_rad': 4.644516}
1  unmatched_submission      NaN          0.0                                ❏
↳
      {}                                                                    ❏

      submission                                                            ❏
0
↳ {}                                                                    ❏
1  {'P_days': 4.232078, 'm_sin_i_mjup': 4.390132, 'e': 0.0, 'omega_rad': 0.0, ❏
↳ 'l_rad': 6.283185}

      guess_index  nearest_truth_index                                ❏
↳
      submission \
0          0          0  {'P_days': 4.232078, 'm_sin_i_mjup': 4.
↳ 390132, 'e': 0.0, 'omega_rad': 0.0, 'l_rad': 6.283185}

```

```

↪ nearest_truth period_rel_error mass_rel_error eccentricity_abs_error ↪
↪ omega_error_rad \
0 {'P_days': 4.230785, 'm_sin_i_mjup': 0.461, 'e': 0.013, 'omega_rad': 1.
↪ 012291, 'l_rad': 4.644516} 0.000306 8.523063 ↪
↪ 0.013 1.012291

l_error_rad
0 1.638669

```

1.3 3. Trajectory Graph and Structural Metrics

Events are reconstructed from the saved planner/engineer/reviewer/executor trace. Edges follow causal order and feedback loops.

```

[4]: def build_events(transitions, qwen_records, executors):
    reason_stop_words = {
        "stop", "end", "ended", "length", "finish", "finished", "none", "null", ↪
↪ "true", "false",
        "verdict", "approve", "approved", "approval", "revise", "revised", ↪
↪ "revision", "code", "result",
        "script", "proposed", "executed", "review", "reviewer", "feedback", ↪
↪ "iteration", "engineer",
        "planner", "executor", "plan", "written", "returned", "before", ↪
↪ "after", "next", "path",
        "characters", "character", "thinking", "process", "analyze", "request", ↪
↪ "evidence",
        "because", "inst", "uniq", "indice", "plausi", "exit", "successfully",
        "the", "a", "an", "and", "or", "to", "of", "in", "for", "with", "by", ↪
↪ "on", "at", "as",
        "is", "are", "was", "were", "be", "been", "being", "this", "that", ↪
↪ "it", "its", "from",
        "when", "during", "while", "which", "will", "shall", "can", "could", ↪
↪ "should", "would",
        "may", "might", "must", "have", "has", "had", "not", "without", ↪
↪ "within", "into", "all",
    }

    def extract_last_meaningful_reason_word(*texts):
        words = []
        for text in texts:
            # Extract semantic reason words source-by-source, not transport ↪
↪ finish reasons or file/path fragments.
            raw_text = str(text or "")
            words = re.findall(r"[A-Za-z]{4,}", raw_text.lower())
            if raw_text.rstrip() and raw_text.rstrip()[-1].isalnum() and ↪
↪ len(raw_text) > 120:

```

```

        words = words[:-1]
        for word in reversed(words):
            if word not in reason_stop_words:
                return word
    return ""

events = []
qwen_by_key = {(r.get("role"), r.get("phase"), r.get("iteration")): r for r in
↳in qwen_records}
executor_by_iteration = {r.get("script_path", ""): r for r in executors}
for idx, tr in enumerate(transitions):
    role = tr.get("role")
    phase = tr.get("phase")
    iteration = tr.get("iteration")
    qwen = qwen_by_key.get((role, "engineering" if phase == "write_code"
↳else phase, iteration), {})
    event = {
        "event_id": f"e{idx:03d}",
        "index": idx,
        "role": role,
        "phase": phase,
        "iteration": iteration,
        "verdict": tr.get("verdict"),
        "finish_reason": tr.get("finish_reason"),
        "reasoning_present": bool(tr.get("reasoning_present")),
        "details": tr.get("details", ""),
        "text_excerpt": str(qwen.get("text", ""))[:500],
        "tokens": qwen.get("tokens", {}),
    }
    reason_sources = [event["details"]]
    if role in {"planner", "reviewer"}:
        reason_sources.append(event["text_excerpt"])
    event["reason_last_word"] =
↳extract_last_meaningful_reason_word(*reason_sources)
    events.append(event)
return events

events = build_events(transition_records, qwen_trace, executor_records)
edges = []
for prev, cur in zip(events, events[1:]):
    edges.append({"source": prev["event_id"], "target": cur["event_id"],
↳"edge_type": "temporal"})
for event in events:
    if event["phase"] == "feedback_to_engineer":
        later = [e for e in events if e["iteration"] > event["iteration"] and
↳e["role"] == "engineer" and e["phase"] == "write_code"]
        if later:

```

```

        edges.append({"source": event["event_id"], "target":
↳later[0]["event_id"], "edge_type": "revision_feedback"})

role_counts = Counter(e["role"] for e in events)
phase_counts = Counter(e["phase"] for e in events)
reason_last_word_counts = Counter(e.get("reason_last_word") for e in events if
↳e.get("reason_last_word"))
revision_events = [e for e in events if e["phase"] == "feedback_to_engineer"]
approve_code_events = [e for e in events if e["verdict"] == "APPROVE_CODE"]
approve_result_events = [e for e in events if e["verdict"] == "APPROVE_RESULT"]

trajectory_graph = {
    "events": events,
    "edges": edges,
    "metrics": {
        "event_count": len(events),
        "edge_count": len(edges),
        "role_counts": dict(role_counts),
        "phase_counts": dict(phase_counts),
        "reason_last_word_counts": dict(reason_last_word_counts),
        "revision_count": len(revision_events),
        "code_review_count": len(code_reviews),
        "result_review_count": len(result_reviews),
        "executor_count": len(executor_records),
        "approve_code_count": len(approve_code_events),
        "approve_result_count": len(approve_result_events),
        "max_iteration_seen": max([e["iteration"] for e in events if
↳isinstance(e.get("iteration"), int)] or [0]),
    },
}
write_json(EVAL_OUT / "trajectory_graph.json", trajectory_graph)
pd.DataFrame(events).to_csv(EVAL_OUT / "trajectory_events.csv", index=False)

display(pd.DataFrame([[k, v] for k, v in trajectory_graph["metrics"].items()],
↳columns=["metric", "value"]))
display(pd.DataFrame(events)[["event_id", "role", "phase", "iteration",
↳"verdict", "reason_last_word", "finish_reason", "details"]])

```

```

metric \
0      event_count
1      edge_count
2      role_counts
3      phase_counts
4  reason_last_word_counts
5      revision_count
6      code_review_count
7      result_review_count

```

```

8         executor_count
9         approve_code_count
10        approve_result_count
11        max_iteration_seen

↵
↵value
0
↵
↵ 40
1
↵
↵ 48
2
↵          {'planner': 1, 'engineer': 10, 'reviewer': 26, 'executor':
↵ 3}
3
↵          {'planning': 1, 'write_code': 10, 'code_review': 10,
↵ 'feedback_to_engineer': 10, 'save_approved_code': 3, 'execute': 3,
↵ 'result_review': 3}
4  {'task': 1, 'environment': 1, 'dataset': 1, 'prevents': 1, 'kepler': 1,
↵ 'passed': 1, 'crash': 1, 'application': 1, 'multi': 1, 'workflow': 3, 'models':
↵ 1, '...
5
↵
↵ 10
6
↵
↵ 10
7
↵
↵ 3
8
↵
↵ 3
9
↵
↵ 6
10
↵
↵ 0
11
↵
↵ 10

event_id      role      phase  iteration      verdict
↵reason_last_word finish_reason \

```

0	e000	planner	planning	0	PLAN_WRITTEN	␣
↪	task	length				
1	e001	engineer	write_code	1	SCRIPT_PROPOSED	␣
↪		stop				
2	e002	reviewer	code_review	1	REVISE_CODE	␣
↪	environment	stop				
3	e003	reviewer	feedback_to_engineer	1	REVISE_CODE	␣
↪		stop				
4	e004	engineer	write_code	2	SCRIPT_PROPOSED	␣
↪		stop				
5	e005	reviewer	code_review	2	REVISE_CODE	␣
↪	dataset	stop				
6	e006	reviewer	feedback_to_engineer	2	REVISE_CODE	␣
↪		stop				
7	e007	engineer	write_code	3	SCRIPT_PROPOSED	␣
↪		stop				
8	e008	reviewer	code_review	3	REVISE_CODE	␣
↪	prevents	stop				
9	e009	reviewer	feedback_to_engineer	3	REVISE_CODE	␣
↪		stop				
10	e010	engineer	write_code	4	SCRIPT_PROPOSED	␣
↪		stop				
11	e011	reviewer	code_review	4	REVISE_CODE	␣
↪	kepler	length				
12	e012	reviewer	feedback_to_engineer	4	REVISE_CODE	␣
↪		length				
13	e013	engineer	write_code	5	SCRIPT_PROPOSED	␣
↪		stop				
14	e014	reviewer	code_review	5	REVISE_CODE	␣
↪	passed	length				
15	e015	reviewer	feedback_to_engineer	5	REVISE_CODE	␣
↪		length				
16	e016	engineer	write_code	6	SCRIPT_PROPOSED	␣
↪		stop				
17	e017	reviewer	code_review	6	REVISE_CODE	␣
↪	crash	length				
18	e018	reviewer	feedback_to_engineer	6	REVISE_CODE	␣
↪		length				
19	e019	engineer	write_code	7	SCRIPT_PROPOSED	␣
↪		stop				
20	e020	reviewer	code_review	7	REVISE_CODE	␣
↪	application	length				
21	e021	reviewer	feedback_to_engineer	7	REVISE_CODE	␣
↪		length				
22	e022	engineer	write_code	8	SCRIPT_PROPOSED	␣
↪		stop				

2 VERDICT: REVISE_CODE - The script writes to `agent\_diagnostics.json` which
↳is not part of the required output contract and may violate the "no extra
↳files" ...

3
↳ code feedback returned before next engineer
↳iteration

4
↳ 6303
↳characters

5 VERDICT: REVISE_CODE - The script fails to handle multi-instrument data
↳offsets, treating all radial velocities as if they share a common zero-point,
↳which ...

6
↳ code feedback returned before next engineer
↳iteration

7
↳ 7790
↳characters

8 VERDICT: REVISE_CODE - The script fails to handle the multi-instrument data
↳correctly by applying a single global `gamma` to all instruments while
↳simultane...

9
↳ code feedback returned before next engineer
↳iteration

10
↳ 9008
↳characters

11 VERDICT: REVISE_CODE - The script fails to handle the `instruments` column
↳correctly in the `residuals` function, causing a shape mismatch between the
↳scala...

12
↳ code feedback returned before next engineer
↳iteration

13
↳ 9277
↳characters

14 VERDICT: REVISE_CODE - The script fails to handle the `instruments` column
↳correctly in the `keplerian\_rv` residual calculation, causing a shape mismatch
↳or...

15
↳ code feedback returned before next engineer
↳iteration

16
↳ 9364
↳characters

```

17  VERDICT: REVISE_CODE - The `keplerian_rv` function fails to handle array
    ↳inputs for the Newton-Raphson solver in Kepler's equation, causing a shape
    ↳mismatch...
18
    ↳                                code feedback returned before next engineer
    ↳iteration
19
    ↳                                10139
    ↳characters
20  VERDICT: REVISE_CODE - The script fails to handle the `instruments` column
    ↳correctly in the final residual calculation (Step 3 diagnostics) and the
    ↳output g...
21
    ↳                                code feedback returned before next engineer
    ↳iteration
22
    ↳                                8552
    ↳characters
23  VERDICT: APPROVE_CODE\n\nThe script implements a scientifically plausible
    ↳radial-velocity analysis pipeline aligned with the planner's steps:\n1.
    ↳**Data Lo...
24
    ↳                                C:
    ↳\Users\Anwender\Science-Work-Flow-\outputs\qwen_saeed_agent_stargazer\agent_workflow\iterati
    ↳py
25
    ↳                                exit_code=0
26  VERDICT: REVISE_RESULT - The submission contains an empty planet table
    ↳("planets": []), failing to identify any radial velocity signals despite the
    ↳planner'...
27
    ↳                                result feedback returned before next engineer
    ↳iteration
28
    ↳                                12076
    ↳characters
29  The engineer's script implements a scientifically plausible radial velocity
    ↳analysis pipeline:\n1.  **Data Loading**: Correctly parses the JSON input and
    ↳ha...
30
    ↳                                C:
    ↳\Users\Anwender\Science-Work-Flow-\outputs\qwen_saeed_agent_stargazer\agent_workflow\iterati
    ↳py
31
    ↳                                exit_code=0

```

```

32 VERDICT: REVISE_RESULT - The submitted planet has an eccentricity of 0.99
   ↳and a mass of 0.0003 M_Jup, which is physically implausible for a
   ↳radial-velocity ...
33
   ↳                                     result feedback returned before next engineer
   ↳iteration
34
   ↳                                     12770
   ↳characters
35 The engineer's script implements a scientifically plausible radial velocity
   ↳analysis pipeline:\n1.  **Data Loading**: Correctly parses the JSON input and
   ↳ha...
36                                     C:
   ↳\Users\Anwender\Science-Work-Flow-\outputs\qwen_saeed_agent_stargazer\agent_workflow\iterati
   ↳py
37
   ↳                                     U
   ↳exit_code=0
38 VERDICT: REVISE_RESULT - The submission contains an empty planet table
   ↳("planets": []), indicating the numerical search failed to identify or report
   ↳any pla...
39
   ↳                                     result feedback returned before next engineer
   ↳iteration

```

1.4 4. Anchor Labels and First Violated Anchor

Anchors are proposal-style intermediate and final checks. They include artifact gates, transition-order gates, reviewer gates, and STARGAZER physical/model anchors.

```

[5]: criteria = benchmark.get("criteria", {})
      component = benchmark.get("component_breakdown", {})
      nearest_rows = benchmark.get("nearest_truth_rows", [])
      nearest = nearest_rows[0] if nearest_rows else {}

      def anchor(name, passed, layer, originating_agent, evidence):
          return {
              "anchor": name,
              "passed": bool(passed),
              "layer": layer,
              "originating_agent": originating_agent,
              "evidence": evidence,
          }

      phases = [(r.get("role"), r.get("phase"), r.get("verdict")) for r in
   ↳transition_records]
      executor_after_code_approval = all(

```

```

    p[1] != "execute" or any(q[1] == "save_approved_code" and q[2] ==
    ↳ "APPROVE_CODE" for q in phases[:i])
    for i, p in enumerate(phases)
)
benchmark_after_result_approval = bool(final_verdict.get("benchmark_ran")) ==
    ↳ bool(final_verdict.get("loop_stop_reason") == "APPROVE_RESULT")

anchors = [
    anchor("planner_present", any(e["role"] == "planner" for e in events),
    ↳ "trajectory", "planner", "planner event exists"),
    anchor("engineer_code_proposed", any(e["role"] == "engineer" and e["phase"]
    ↳ == "write_code" for e in events), "trajectory", "engineer", "engineer
    ↳ write_code events exist"),
    anchor("reviewer_code_review_before_execution",
    ↳ executor_after_code_approval, "trajectory", "reviewer", "executor appears
    ↳ only after APPROVE_CODE"),
    anchor("executor_success", all(r.get("exit_code") == 0 for r in
    ↳ executor_records) and bool(executor_records), "artifact", "executor",
    ↳ f"executor_count={len(executor_records)}"),
    anchor("agent_submission_evidence_available", submission_artifact_available
    ↳ or benchmark.get("evaluable", False), "artifact", "executor/evaluator",
    ↳ f"path_exists={submission_artifact_available}"),
    ↳ benchmark_source={benchmark_source}"),
    anchor("saved_benchmark_record_available", benchmark.get("evaluable",
    ↳ False), "artifact", "evaluator", f"benchmark_source={benchmark_source}"),
    anchor("result_review_before_benchmark", benchmark_after_result_approval,
    ↳ "trajectory", "reviewer", f"loop_stop_reason={final_verdict.
    ↳ get('loop_stop_reason')}"; benchmark_ran={final_verdict.
    ↳ get('benchmark_ran')}"),
    anchor("benchmark_evaluable", benchmark.get("evaluable", False), "science",
    ↳ "evaluator", f"evaluable={benchmark.get('evaluable')}"),
    anchor("planet_count_matches", criteria.get("planet_count_matches", False),
    ↳ "science", "engineer", f"submitted={benchmark.get('submitted_planet_count')}";
    ↳ truth={benchmark.get('truth_planet_count')}"),
    anchor("hungarian_match_positive", criteria.
    ↳ get("matched_truth_fraction_positive", False), "science", "engineer/
    ↳ reviewer", f"matched_truth_fraction={benchmark.
    ↳ get('matched_truth_fraction')}"),
    anchor("match_score_at_least_0_8", criteria.get("match_score_at_least_0_8",
    ↳ False), "science", "engineer/reviewer", f"match_score={benchmark.
    ↳ get('match_score')}"),
    anchor("delta_bic_positive", criteria.get("delta_bic_positive", False),
    ↳ "science", "engineer", f"delta_bic_per_point={benchmark.
    ↳ get('delta_bic_per_point')}"),

```

```

    anchor("nearest_period_recovered", nearest.get("period_rel_error", math.
↪inf) <= 0.10 if nearest else False, "science", "engineer",
↪f"nearest_period_rel_error={nearest.get('period_rel_error')}",
    anchor("nearest_mass_reasonable", nearest.get("mass_rel_error", math.inf)
↪<= 0.50 if nearest else False, "science", "engineer",
↪f"nearest_mass_rel_error={nearest.get('mass_rel_error')}",
    anchor("nearest_phase_reasonable", (nearest.get("omega_error_rad", math.
↪inf) <= 1.0 and nearest.get("l_error_rad", math.inf) <= 1.0) if nearest else
↪False, "science", "engineer", f"omega_error={nearest.get('omega_error_rad')};
↪l_error={nearest.get('l_error_rad')}"),
    anchor("reviewer_rejected_bad_result_before_final", not (benchmark.
↪get("evaluatable") and not benchmark.get("passed") and final_verdict.
↪get("loop_stop_reason") == "APPROVE_RESULT"), "trajectory", "reviewer",
↪"APPROVE_RESULT should not coincide with failed STARGAZER anchors"),
]

first_failed = next((item for item in anchors if not item["passed"]), {"anchor":
↪ "none", "passed": True, "layer": "none", "originating_agent": "none",
↪ "evidence": "all anchors passed"})
anchor_score = sum(1 for item in anchors if item["passed"]) / len(anchors)

anchor_report = {"anchors": anchors, "first_violated_anchor": first_failed,
↪ "anchor_score": round(anchor_score, 6)}
write_json(EVAL_OUT / "anchor_report.json", anchor_report)

display(pd.DataFrame(anchors))
display(Markdown(f"First violated anchor: **{first_failed['anchor']}** at layer
↪ **{first_failed['layer']}**, attributed to
↪ **{first_failed['originating_agent']}**."))

```

	anchor	passed	layer	
↪originating_agent				↪
↪evidence				
0	planner_present	True	trajectory	↪
↪planner				planner↪
↪event exists				
1	engineer_code_proposed	True	trajectory	↪
↪engineer				engineer write_code↪
↪events exist				
2	reviewer_code_review_before_execution	True	trajectory	↪
↪reviewer				executor appears only after↪
↪APPROVE_CODE				
3	executor_success	True	artifact	↪
↪executor				↪
↪executor_count=3				

```

4      agent_submission_evidence_available      True      artifact  executor/
  ↪evaluator  path_exists=False;␣
  ↪benchmark_source=saved_benchmark_json_submission_file_missing
5      saved_benchmark_record_available      True      artifact
  ↪evaluator
  ↪benchmark_source=saved_benchmark_json_submission_file_missing
6      result_review_before_benchmark      True      trajectory
  ↪reviewer
  ↪benchmark_ran=True
  ↪loop_stop_reason=APPROVE_RESULT;␣
7      benchmark_evaluable      True      science
  ↪evaluator
  ↪evaluable=True
8      planet_count_matches      True      science
  ↪engineer
  ↪submitted=1; truth=1
9      hungarian_match_positive      False      science  engineer/
  ↪reviewer
  ↪matched_truth_fraction=0.0
10     match_score_at_least_0_8      False      science  engineer/
  ↪reviewer
  ↪match_score=0.0
11     delta_bic_positive      False      science
  ↪engineer
  ↪delta_bic_per_point=-418569.316306
12     nearest_period_recovered      True      science
  ↪engineer
  ↪nearest_period_rel_error=0.000306
13     nearest_mass_reasonable      False      science
  ↪engineer
  ↪nearest_mass_rel_error=8.523063
14     nearest_phase_reasonable      False      science
  ↪engineer
  ↪l_error=1.638669
  ↪omega_error=1.012291;␣
15  reviewer_rejected_bad_result_before_final      False      trajectory
  ↪reviewer
  ↪STARGAZER anchors
  ↪APPROVE_RESULT should not coincide with failed␣

```

First violated anchor: **hungarian_match_positive** at layer **science**, attributed to **engineer/reviewer**.

1.5 5. Failure Taxonomy Detectors

These detectors implement the proposal's O2 taxonomy over the trajectory graph plus anchor labels.

```

[6]: labels = []
failed_components = set(component.get("failed_components", []))
review_text = "\n".join(str(r.get("text", "")) for r in result_reviews).lower()

```

```

code_review_text = "\n".join(str(r.get("text", "")) for r in code_reviews).
    ↪lower()

if not benchmark.get("evaluable", False):
    labels.append("format_fragility")
if benchmark.get("evaluable", False) and not benchmark.get("passed", False):
    labels.append("scientific_anchor_failure")
if "period_recovery" in failed_components and nearest.get("period_rel_error",
    ↪math.inf) > 0.10:
    labels.append("alias_convergence_or_missed_period")
if "mass_amplitude" in failed_components or (nearest and nearest.
    ↪get("mass_rel_error", math.inf) > 0.50):
    labels.append("mass_amplitude_mismatch")
if "phase_or_eccentricity" in failed_components:
    labels.append("phase_parameter_mismatch")
if "model_fit" in failed_components:
    labels.append("model_fit_degradation")
if final_verdict.get("loop_stop_reason") == "APPROVE_RESULT" and benchmark.
    ↪get("evaluable") and not benchmark.get("passed"):
    labels.append("critic_masking")
    labels.append("silent_acceptance")
if len(revision_events) >= 4:
    labels.append("perseveration_or_retry_loop")
if "lomb" in code_review_text and code_review_text.count("tau") >= 2:
    labels.append("periodogram_implementation_instability")
if len(role_counts) < 4 or role_counts.get("executor", 0) == 0:
    labels.append("coordination_collapse")
if any(e.get("finish_reason") == "length" for e in events):
    labels.append("truncated_reasoning_or_output")

detector_report = {
    "detectors_fired": sorted(set(labels)) if labels else
    ↪["no_failure_detected"],
    "first_violated_anchor": first_failed,
    "originating_agent": first_failed.get("originating_agent"),
    "interpretation": (
        "The workflow produced and reviewer-approved an executor artifact, but
    ↪STARGAZER physical/model anchors failed. "
        "This is a proposal-relevant silent acceptance / critic-masking case,
    ↪not an environment failure."
        if "critic_masking" in labels else
        "No critic-masking failure detected by the configured trajectory labels.
    ↪"
    ),
}
write_json(EVAL_OUT / "detector_report.json", detector_report)

```

```
display(pd.DataFrame([[label] for label in detector_report["detectors_fired"]],
↳columns=["detector"]))
display(Markdown(detector_report["interpretation"]))
```

```

              detector
0              critic_masking
1      mass_amplitude_mismatch
2      model_fit_degradation
3  perseveration_or_retry_loop
4      phase_parameter_mismatch
5      scientific_anchor_failure
6              silent_acceptance
7  truncated_reasoning_or_output
```

The workflow produced and reviewer-approved an executor artifact, but STARGAZER physical/model anchors failed. This is a proposal-relevant silent acceptance / critic-masking case, not an environment failure.

1.6 6. Proposal-Aligned Objective Evaluation

This table states what this single run can and cannot support for O1/O2/O3.

```
[7]: objective_rows = [
    {
        "objective": "O1 localisation infrastructure",
        "status": "partial_support",
        "evidence": f"trace events={len(events)}, edges={len(edges)},
↳first_failed_anchor={first_failed['anchor']},
↳originating_agent={first_failed['originating_agent']}",
        "limitation": "Single run; no manual-label precision/recall; event
↳graph is reconstructed from notebook logs, not a framework-level
↳non-invasive observer.",
    },
    {
        "objective": "O2 failure taxonomy and detectors",
        "status": "partial_support",
        "evidence": ", ".join(detector_report["detectors_fired"]),
        "limitation": "Detector labels are rule-based for one STARGAZER run; no
↳validation set, no precision/recall/F1.",
    },
    {
        "objective": "O3 early prediction across verification regimes",
        "status": "not_tested",
        "evidence": f"revision_count={len(revision_events)} and code/result
↳review failures occurred before final approval",
        "limitation": "No paired stress levels, no Lean comparison, no
↳bootstrap statistics.",
    },
]
```



```

    },
    {
        "objective": "STARGAZER scientific correctness",
        "status": "scientific_fail",
        "evidence": f"passed={benchmark.get('passed')}, match_score={benchmark.
↪get('match_score')}, rms={benchmark.get('rms')},
↪failed_components={component.get('failed_components')}",
        "limitation": "Period was near the truth in nearest-period comparison,
↪but Hungarian match, mass/phase/model-fit anchors failed.",
    },
    {
        "objective": "Silent failure / critic-masking evidence",
        "status": "supported_for_this_run",
        "evidence": f"reviewer returned APPROVE_RESULT while benchmark
↪passed={benchmark.get('passed')}",
        "limitation": "This is a single observed instance; not a rate estimate.
↪",
    },
]
objective_df = pd.DataFrame(objective_rows)
objective_df.to_csv(EVAL_OUT / "proposal_objective_evaluation.csv", index=False)
display(objective_df)

```

	objective	status \
0	01 localisation infrastructure	partial_support
1	02 failure taxonomy and detectors	partial_support
2	03 early prediction across verification regimes	not_tested
3	STARGAZER scientific correctness	scientific_fail
4	Silent failure / critic-masking evidence	supported_for_this_run


```

↪
↪evidence \
0
↪trace events=40, edges=48,
↪first_failed_anchor=hungarian_match_positive, originating_agent=engineer/
↪reviewer
1 critic_masking, mass_amplitude_mismatch, model_fit_degradation,
↪perseveration_or_retry_loop, phase_parameter_mismatch,
↪scientific_anchor_failure, silent_acc...
2
↪revision_count=10 and code/result review failures occurred before final
↪approval
3
↪passed=False, match_score=0.0, rms=354.505911,
↪failed_components=['mass_amplitude', 'model_fit', 'period_recovery',
↪'phase_or_eccentricity']

```

```

4
↪ reviewer returned APPROVE_RESULT while benchmark
↪ passed=False

↪ limitation
0 Single run; no manual-label precision/recall; event graph is reconstructed
↪ from notebook logs, not a framework-level non-invasive observer.
1 Detector labels are rule-based for
↪ one STARGAZER run; no validation set, no precision/recall/F1.
2 No
↪ paired stress levels, no Lean comparison, no bootstrap statistics.
3 Period was near the truth in nearest-period
↪ comparison, but Hungarian match, mass/phase/model-fit anchors failed.
4
↪ This is a single observed instance; not a rate estimate.

```

1.7 7. Minimum Evidence Table

This follows the project report frame: each row states status, evidence, and limitation.

```

[8]: minimum_rows = [
    ["01 trace schema/dashboard/taxonomy", "partial", f"trajectory_graph.json
    ↪ with {len(events)} events and {len(edges)} edges", "Notebook reconstruction,
    ↪ not production dashboard"],
    ["01 first failed anchor localisation", "partial",
    ↪ f"{first_failed['anchor']} attributed to
    ↪ {first_failed['originating_agent']}", "Automatic anchor rule, not
    ↪ manual-labelled precision/recall"],
    ["02 STARGAZER partially verifiable anchors", "available",
    ↪ f"score={benchmark.get('score')}; failed={component.
    ↪ get('failed_components')}", "Only one real task"],
    ["02 Lean fully step-verifiable anchors", "not_tested", "No Lean run in
    ↪ this notebook", "Transfer claim unavailable"],
    ["02 detector validation against manual labels", "not_tested", "", "
    ↪ join(detector_report["detectors_fired"]), "No labelled corpus"],
    ["03 paired/bootstrap comparison", "not_tested", "No paired baseline/stress
    ↪ suite", "No p-values/effect sizes"],
    ["03 early-warning difficulty degradation", "not_tested",
    ↪ f"revision_count={len(revision_events)}", "No difficulty series"],
    ["Langfuse/open-source observability export", "not_available", "Local JSON/
    ↪ CSV artifacts only", "No external observability export"],
]
minimum_df = pd.DataFrame(minimum_rows, columns=["evidence_row", "status",
    ↪ "evidence", "limitation"])
minimum_df.to_csv(EVAL_OUT / "minimum_evidence_table.csv", index=False)
display(minimum_df)

```

	evidence_row	status \
0	01 trace schema/dashboard/taxonomy	partial
1	01 first failed anchor localisation	partial
2	02 STARGAZER partially verifiable anchors	available
3	02 Lean fully step-verifiable anchors	not_tested
4	02 detector validation against manual labels	not_tested
5	03 paired/bootstrap comparison	not_tested
6	03 early-warning difficulty degradation	not_tested
7	Langfuse/open-source observability export	not_available

↪		↪	
↪evidence \			
0			
↪	trajectory_graph.json with 40 events and 48		
↪edges			
1			
↪	hungarian_match_positive attributed to engineer/		
↪reviewer			
2		score=0.4;	
↪failed=['mass_amplitude', 'model_fit', 'period_recovery',			
↪'phase_or_eccentricity']			
3			
↪	No Lean run in this		
↪notebook			
4	critic_masking, mass_amplitude_mismatch, model_fit_degradation,		
↪perseveration_or_retry_loop, phase_parameter_mismatch,			
↪scientific_anchor_failure, silent_acc...			
5			
↪	No paired baseline/stress		
↪suite			
6			
↪			
↪revision_count=10			
7			
↪	Local JSON/CSV artifacts		
↪only			

	limitation
0	Notebook reconstruction, not production dashboard
1	Automatic anchor rule, not manual-labelled precision/recall
2	Only one real task
3	Transfer claim unavailable
4	No labelled corpus
5	No p-values/effect sizes
6	No difficulty series
7	No external observability export

1.8 8. Real vs Agent-Proposed Planet Visualization

This evaluation-only cell uses the hidden truth planet and the agent-produced submission to draw a compact visual comparison. The agent workflow itself did not receive the hidden truth.

```
[9]: from IPython.display import HTML
import math

truth_planets = benchmark.get("truth_planets", [])
submitted_planets = benchmark.get("submitted_planets", [])
assert truth_planets and submitted_planets, "Need both truth and submitted_
    ↪ planets to draw comparison."

truth_planet = truth_planets[0]
agent_planet = submitted_planets[0]

def orbit_points(planet, n=240):
    P = float(planet.get("P_days", 1.0))
    e = max(0.0, min(float(planet.get("e", 0.0) or 0.0), 0.95))
    omega = float(planet.get("omega_rad", 0.0) or 0.0)
    a = P ** (2.0 / 3.0)
    points = []
    for i in range(n + 1):
        nu = 2.0 * math.pi * i / n
        r = a * (1.0 - e * e) / (1.0 + e * math.cos(nu))
        points.append((r * math.cos(nu + omega), r * math.sin(nu + omega)))
    return points, a

def planet_marker(planet):
    P = float(planet.get("P_days", 1.0))
    e = max(0.0, min(float(planet.get("e", 0.0) or 0.0), 0.95))
    omega = float(planet.get("omega_rad", 0.0) or 0.0)
    l = float(planet.get("l_rad", 0.0) or 0.0)
    M = (1 - omega) % (2.0 * math.pi)
    E = M
    for _ in range(20):
        E -= (E - e * math.sin(E) - M) / max(1e-9, 1.0 - e * math.cos(E))
    nu = 2.0 * math.atan2(math.sqrt(1.0 + e) * math.sin(E / 2.0), math.sqrt(1.0
    ↪ - e) * math.cos(E / 2.0))
    a = P ** (2.0 / 3.0)
    r = a * (1.0 - e * e) / (1.0 + e * math.cos(nu))
    return r * math.cos(nu + omega), r * math.sin(nu + omega)

truth_points, truth_a = orbit_points(truth_planet)
agent_points, agent_a = orbit_points(agent_planet)
all_points = truth_points + agent_points
scale = max(max(abs(x), abs(y)) for x, y in all_points) or 1.0
tx, ty = planet_marker(truth_planet)
```

```

ax, ay = planet_marker(agent_planet)

fields = ["P_days", "m_sin_i_mjup", "e", "omega_rad", "l_rad"]
truth_values = [float(truth_planet.get(field, 0.0) or 0.0) for field in fields]
agent_values = [float(agent_planet.get(field, 0.0) or 0.0) for field in fields]

def svg_polyline(points, cx, cy, radius):
    return " ".join(f"{cx + (x / scale) * radius:.2f},{cy - (y / scale) * \
radius:.2f}" for x, y in points)

def svg_text(x, y, text, size=13, weight="400", fill="#243040", anchor="start"):
    return f'<text x="{x}" y="{y}" font-size="{size}" font-family="Segoe UI,\
Arial, sans-serif" font-weight="{weight}" fill="{fill}" \
text-anchor="{anchor}">{text}</text>'

chart_max = max(truth_values + agent_values + [1.0])
bar_parts = []
bar_x0, bar_y0 = 560, 360
bar_w, gap = 18, 58
for i, field in enumerate(fields):
    x = bar_x0 + i * gap
    th = 250.0 * truth_values[i] / chart_max
    ah = 250.0 * agent_values[i] / chart_max
    bar_parts.append(f'<rect x="{x}" y="{bar_y0 - th:.2f}" width="{bar_w}" \
height="{th:.2f}" fill="#1f77b4"/>')
    bar_parts.append(f'<rect x="{x + bar_w + 3}" y="{bar_y0 - ah:.2f}" \
width="{bar_w}" height="{ah:.2f}" fill="#d62728"/>')
    bar_parts.append(svg_text(x + 12, 385, field, size=11, anchor="middle"))

svg = f'''
<svg xmlns="http://www.w3.org/2000/svg" width="1040" height="460" viewBox="0 0 \
1040 460">
  <rect width="1040" height="460" fill="#ffffff"/>
  {svg_text(40, 34, "Real vs agent-proposed planet", 22, "700")}
  {svg_text(40, 60, "Orbit sketch is normalized; bars show raw submitted fields.\
", 13, "400", "#5b6778")}
  <rect x="35" y="82" width="465" height="330" fill="#f8fafc" stroke="#d5dbe3"/>
  <polyline points="{svg_polyline(truth_points, 267, 247, 145)}" fill="none" \
stroke="#1f77b4" stroke-width="3"/>
  <polyline points="{svg_polyline(agent_points, 267, 247, 145)}" fill="none" \
stroke="#d62728" stroke-width="3" stroke-dasharray="8 6"/>
  <circle cx="267" cy="247" r="9" fill="#f2b01e" stroke="#111827"/>
  <circle cx="{267 + (tx / scale) * 145:.2f}" cy="{247 - (ty / scale) * 145:\
2f}" r="7" fill="#1f77b4" stroke="#ffffff" stroke-width="2"/>
  <circle cx="{267 + (ax / scale) * 145:.2f}" cy="{247 - (ay / scale) * 145:\
2f}" r="7" fill="#d62728" stroke="#ffffff" stroke-width="2"/>

```

```

{svg_text(55, 108, "orbit geometry", 15, "700")}
<line x1="365" y1="105" x2="395" y2="105" stroke="#1f77b4" stroke-width="3"/
↪>{svg_text(402, 110, "truth", 12)}
<line x1="365" y1="125" x2="395" y2="125" stroke="#d62728" stroke-width="3"
↪stroke-dasharray="8 6"/>{svg_text(402, 130, "agent", 12)}
<rect x="535" y="82" width="465" height="330" fill="#f8fafc" stroke="#d5dbe3"/
↪>
{svg_text(555, 108, "parameter comparison", 15, "700")}
<line x1="555" y1="360" x2="955" y2="360" stroke="#aab4c0"/>
{''.join(bar_parts)}
<rect x="830" y="110" width="14" height="14" fill="#1f77b4"/>{svg_text(850,
↪122, "truth", 12)}
<rect x="830" y="132" width="14" height="14" fill="#d62728"/>{svg_text(850,
↪144, "agent", 12)}
</svg>
'''

```

```

visual_path = EVAL_OUT / "real_vs_agent_planet_visualization.svg"
visual_path.write_text(svg, encoding="utf-8")
display(HTML(svg))

visual_rows = []
for field, truth_value, agent_value in zip(fields, truth_values, agent_values):
    visual_rows.append({
        "field": field,
        "truth": truth_value,
        "agent": agent_value,
        "relative_error": relative_error(agent_value, truth_value) if field not
↪in ["omega_rad", "l_rad"] else None,
        "angle_error_rad": angle_distance(agent_value, truth_value) if field in
↪["omega_rad", "l_rad"] else None,
    })
visual_df = pd.DataFrame(visual_rows)
visual_df.to_csv(EVAL_OUT / "real_vs_agent_planet_parameters.csv", index=False)
display(visual_df)
display(Markdown(f"Saved visualization to `{visual_path}`."))

```

<IPython.core.display.HTML object>

	field	truth	agent	relative_error	angle_error_rad
0	P_days	4.230785	4.232078e+00	0.000306	NaN
1	m_sin_i_mjup	0.461000	4.390132e+00	8.523063	NaN
2	e	0.013000	2.699629e-15	1.000000	NaN
3	omega_rad	1.012291	0.000000e+00	NaN	1.012291
4	l_rad	4.644516	6.283185e+00	NaN	1.638669

Saved visualization to C:\Users\Anwender\Science-Work-Flow-\outputs\qwen_saeed_stargazer_eval\real_

1.9 9. Final Trajectory-Level Evaluation

This is the proposal-aligned conclusion for the run.

```
[10]: status = "scientific_pass" if benchmark.get("passed") else "scientific_fail"
period_note = ""
if nearest:
    period_note = f"Nearest-period comparison shows period_rel_error={nearest.
    ↪get('period_rel_error')}, but mass_rel_error={nearest.get('mass_rel_error')}
    ↪and phase/model anchors fail."

final_report = {
    "run": "qwen_saeed_agent_stargazer",
    "evaluation_type": "trajectory_level_stargazer_single_run",
    "proposal_alignment": {
        "research_question": "Can trajectory signals localise and detect
        ↪failures that output-only evaluation misses in a partially verifiable
        ↪STARGAZER task?",
        "01": objective_rows[0],
        "02": objective_rows[1],
        "03": objective_rows[2],
    },
    "scientific_status": status,
    "first_violated_anchor": first_failed,
    "detectors_fired": detector_report["detectors_fired"],
    "benchmark_summary": {
        "passed": benchmark.get("passed"),
        "score": benchmark.get("score"),
        "match_score": benchmark.get("match_score"),
        "rms": benchmark.get("rms"),
        "reward": benchmark.get("reward"),
        "failed_components": component.get("failed_components"),
    },
    "interpretation": (
        "The run is useful 01/02 evidence for trajectory-level failure
        ↪localisation because the reviewer-approved final result fails hidden
        ↪STARGAZER physical/model anchors. "
        + period_note
        + " The result should not be reported as a scientific success or
        ↪architecture-level claim."
    ),
    "artifacts": {
        "evaluation_dir": str(EVAL_OUT),
        "trajectory_graph": str(EVAL_OUT / "trajectory_graph.json"),
        "anchor_report": str(EVAL_OUT / "anchor_report.json"),
        "detector_report": str(EVAL_OUT / "detector_report.json"),
        "local_benchmark": str(EVAL_OUT / "local_stargazer_benchmark.json"),
    },
}
```

```

}
write_json(EVAL_OUT / "qwen_saeed_stargazer_trajectory_evaluation.json",
↪final_report)

display(Markdown(
    f"Decision: {status}. "
    f"First violated anchor: **{first_failed['anchor']}**. "
    f"Detectors: **{'', '.join(detector_report['detectors_fired'])}**. "
    f"{period_note}"
))
print(json.dumps(final_report, indent=2))

```

Decision: scientific_fail. First violated anchor: hungarian_match_positive. Detectors: critic_masking, mass_amplitude_mismatch, model_fit_degradation, perseveration_or_retry_loop, phase_parameter_mismatch, scientific_anchor_failure, silent_acceptance, truncated_reasoning_or_output. Nearest-period comparison shows period_rel_error=0.000306, but mass_rel_error=8.523063 and phase/model anchors fail.

```

{
  "run": "qwen_saeed_agent_stargazer",
  "evaluation_type": "trajectory_level_stargazer_single_run",
  "proposal_alignment": {
    "research_question": "Can trajectory signals localise and detect failures
that output-only evaluation misses in a partially verifiable STARGAZER task?",
    "01": {
      "objective": "01 localisation infrastructure",
      "status": "partial_support",
      "evidence": "trace events=40, edges=48,
first_failed_anchor=hungarian_match_positive,
originating_agent=engineer/reviewer",
      "limitation": "Single run; no manual-label precision/recall; event graph
is reconstructed from notebook logs, not a framework-level non-invasive
observer."
    },
    "02": {
      "objective": "02 failure taxonomy and detectors",
      "status": "partial_support",
      "evidence": "critic_masking, mass_amplitude_mismatch,
model_fit_degradation, perseveration_or_retry_loop, phase_parameter_mismatch,
scientific_anchor_failure, silent_acceptance, truncated_reasoning_or_output",
      "limitation": "Detector labels are rule-based for one STARGAZER run; no
validation set, no precision/recall/F1."
    },
    "03": {
      "objective": "03 early prediction across verification regimes",
      "status": "not_tested",
      "evidence": "revision_count=10 and code/result review failures occurred
before final approval",

```



```

    "limitation": "No paired stress levels, no Lean comparison, no bootstrap
statistics."
  },
  "scientific_status": "scientific_fail",
  "first_violated_anchor": {
    "anchor": "hungarian_match_positive",
    "passed": false,
    "layer": "science",
    "originating_agent": "engineer/reviewer",
    "evidence": "matched_truth_fraction=0.0"
  },
  "detectors_fired": [
    "critic_masking",
    "mass_amplitude_mismatch",
    "model_fit_degradation",
    "perseveration_or_retry_loop",
    "phase_parameter_mismatch",
    "scientific_anchor_failure",
    "silent_acceptance",
    "truncated_reasoning_or_output"
  ],
  "benchmark_summary": {
    "passed": false,
    "score": 0.4,
    "match_score": 0.0,
    "rms": 354.505911,
    "reward": -335387.464489,
    "failed_components": [
      "mass_amplitude",
      "model_fit",
      "period_recovery",
      "phase_or_eccentricity"
    ]
  },
  "interpretation": "The run is useful 01/02 evidence for trajectory-level
failure localisation because the reviewer-approved final result fails hidden
STARGAZER physical/model anchors. Nearest-period comparison shows
period_rel_error=0.000306, but mass_rel_error=8.523063 and phase/model anchors
fail. The result should not be reported as a scientific success or architecture-
level claim.",
  "artifacts": {
    "evaluation_dir": "C:\\Users\\Anwender\\Science-Work-
Flow-\\outputs\\qwen_saeed_stargazer_eval",
    "trajectory_graph": "C:\\Users\\Anwender\\Science-Work-
Flow-\\outputs\\qwen_saeed_stargazer_eval\\trajectory_graph.json",
    "anchor_report": "C:\\Users\\Anwender\\Science-Work-
Flow-\\outputs\\qwen_saeed_stargazer_eval\\anchor_report.json",

```

```
    "detector_report": "C:\\Users\\Anwender\\Science-Work-  
Flow-\\outputs\\qwen_saeed_stargazer_eval\\detector_report.json",  
    "local_benchmark": "C:\\Users\\Anwender\\Science-Work-  
Flow-\\outputs\\qwen_saeed_stargazer_eval\\local_stargazer_benchmark.json"  
  }  
}
```